ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Subtyping — Class Challenge Set

Part A — Predicting subtyping

Challenge 1. For each of the following pairs of types, decide whether $S <: T$, $T <: S$, both, or neither. Justify each answer in one sentence using the rules from the lecture. Assume $\mathsf{Cat} <: \mathsf{Animal}$ and recall that `Nothing` is the bottom type.

S	T	$S \; ? \; T$	**Justify**
$\{\mathsf{x} : \mathsf{Int}, \mathsf{y} : \mathsf{Int}\}$	$\{\mathsf{x} : \mathsf{Int}\}$		
$\{\mathsf{p} : \mathsf{Int} \to \mathsf{Int}\}$	$\{\mathsf{p} : \mathsf{Int} \to \mathsf{Int}, \mathsf{q} : \mathsf{Bool}\}$		
$\mathsf{Int} \to \mathsf{Int}$	$\mathsf{Int} \to \mathsf{Any}$		
$\mathsf{Any} \to (\mathsf{Any} \to \mathsf{Int})$	$\mathsf{Any} \to (\mathsf{Int} \to \mathsf{Int})$		
$(\mathsf{Any} \to \mathsf{Any}) \to \mathsf{Int}$	$(\mathsf{Any} \to \mathsf{Int}) \to \mathsf{Int}$		
$\{\mathsf{f} : \mathsf{Animal} \to \mathsf{Int}\}$	$\{\mathsf{f} : \mathsf{Cat} \to \mathsf{Int}\}$		
$\{\mathsf{g} : \mathsf{Int} \to \mathsf{Int}\}$	$\{\mathsf{g} : \mathsf{Int} \to \mathsf{Nothing}\}$		

Challenge 2. Consider the following Scala definition:

```
def apply (x:Int, f: Int => Int): Int = f(x)
```

For each of the following higher-order Scala function applications, decide whether the call type-checks. Use the function-subtyping rule.

Application	**Type checks?**
`apply(5, (x: Int) => x + 1)`	
`apply(5, (x: Any) => x.hashCode)`	
`apply(5, (x: Int) => x.toString)`	
`apply(5, (x: Nothing) => x)`	

For each: which direction of the subtyping rule is at play, and why does it (or doesn't it) work?

Part B — Designing rich interfaces

Challenge 3. Design a Scala trait `Container[A]` with the following requirements:

- Implementers should provide *three* abstract methods: `add: A => Container[A]`, `toList: List[A]`, `fromList: List[A] => Container[A]`
- Clients should have access to a rich interface:
- `size: Int`,
- `isEmpty: Boolean`,
- `contains(x: A): Boolean`,
- `concat(other: Container[A]): Container[A]`,

Define the trait with these four concrete methods implemented on top of the two abstract ones.

Now let's extend the trait with a further concrete method `map[B](f: A => B): Container[B]`.

- are the three abstract methods expressive enough to define `map` with the required signature?
- If yes, implement `map` as specified;
- Otherwise upgrage the abstract methods as you see fit, and again implement `map` as specified

Finally implement a concrete subclass `ListContainer[A]` that satisfies the contract.

Key decisiions:

- The four concrete methods all rely on `toList` to do their work. What is the price of this design? (Hint: think about the cost of `size`.) Is there a way to avoid it?
- Can `concat` be implemented without `toList` if we add another abstract method? Which one?

Challenge 4. Consider the following two Scala designs for an interface representing "things that can be compared for equality":

```
// Design 1: subtyping
trait Equatable:
  def equalsTo(that: Equatable): Boolean

// Design 2: parametric
trait Equatable[A]:
  def equalsTo(that: A): Boolean
```

a. Which design lets a class declare that "two `Person`s are equal iff their names match" *without* allowing nonsense like comparing a `Person` to a `Car`?

b. Extend design 2 by replacing `equalsTo` with `compareTo: A => Int` (returning a negative/zero/positive integer, in the style of Java's Comparable). Try to implement `<`, `<=`, and `>=` as concrete methods on the trait, defined in terms of `compareTo`.

What happens when you try to compile it?

What needs to be added to make it work — and what does this tell you about how subtyping interacts with parametric types in interface design?

Part C — Override and substitution

Challenge 5. Consider the following trait and implementations:

```
sealed trait Note:
  def text: String
  def header: String = "Note"
  def render: String = s"[$header]\n$text"

class Reminder(val text: String, val date: String) extends Note:
  override def header: String = s"Reminder for $date"

class Quote(val text: String, val author: String) extends Note:
  override def header: String = s"Quote by $author"
  override def render: String = s"\"$text\"\n  — $author"

class PlainNote(val text: String) extends Note
```

For each of the following expressions, predict the result *and* identify the rule that determines it (static dispatch? dynamic dispatch? inheritance of a default?):

Expression	Result	Determing rule
`(new Reminder("buy milk", "Mon")).header`		
`(new Reminder("buy milk", "Mon")).render`		
`(new Quote("less is more", "Mies")).render`		
`(new PlainNote("hello")).render`		
`(new Note { val text = "x" }).header`		

Food for further thinking:

- Does the last one even compile? if not, why?
- In the second expression, `render` is inherited from `Note`, but the `header` it embeds is dispatched dynamically to `Reminder`'s override. Trace the call carefully.

Challenge 6. Suppose we add the following method to `Note`:

```
def combinedWith(other: Note): String =
  s"${this.render}\n---\n${other.render}"
```

What does `(new Reminder("buy milk", "Mon")).combinedWith(new Quote("less is more", "Mies"))` produce?

Walk through the dispatching of *each* call to `render` in the body of `combinedWith`.

There are two: one on `this`, one on `other`. Both go through dynamic dispatch; explain why this is necessary for the substitution principle.